

Inference-Time Scaffolding for Small-Model Math Reasoning: It’s the Termination, Not the Tools

john.hao.fu@gmail.com

Abstract

We study what actually drives the gains of multi-tool inference-time scaffolding on small math LMs. On GSM8K, wrapping a 2B-parameter Qwen3.5 student (already trained with on-policy distillation and a GRPO variant to 78.47%) in a two-tool agent harness—a Python interpreter and an inversion-style validator—lifts accuracy to 86.05% (+7.58pp); applying the same harness to the 4B teacher lifts it from 83.62% to 93.86% (+10.24pp). With $n=1319$ and binomial standard error $\approx 0.95\text{pp}$, we run a clean ablation and find: removing the Python tool changes accuracy by +0.53pp ($< 1\sigma$), removing the validator changes it by +0.15pp ($< 1\sigma$), but removing a single ~ 80 -token *post-validation nudge* that forces termination drops accuracy by 4.25pp ($> 2\sigma$). We cannot reject the null that either tool individually contributes nothing, yet the nudge contribution is robustly significant. A natural structural explanation is that the validator’s `verification_code` parameter executes Python under the hood, so the two tools function as redundant Python-execution channels rather than as distinct capabilities. Subject to the caveat that we do not isolate a no-tool, nudge-only configuration, we argue that future scaffolding work should treat termination control as a first-class design dimension and report the trivial “remove this tool” baseline. Our findings are consistent with concurrent scaling-law-level results on Effective Feedback Compute [Zhang et al., 2026]: the bottleneck is not the raw compute the harness spends, but the fraction of that compute that produces feedback the agent retains and acts upon.

1 Introduction

Recent work on small-LM math reasoning has converged on a recipe of distillation/RL followed by inference-time tool use: Program-of-Thought [Chen et al., 2022], ToRA [Gou et al., 2023], ReST-style self-improvement, and various “agent” wrappers around chat-format LMs all combine a Python interpreter with some form of verification or self-reflection. Reported gains over chain-of-thought (CoT) baselines are large (+5 to +15pp on GSM8K), and the dominant narrative attributes these gains to the *tools*—to Python’s exactness, or to the verifier’s catching of arithmetic errors.

This paper questions that attribution. We build a deliberately simple two-tool harness—`python` + an inversion-checking `validate` tool—on top of an already-distilled 2B model, observe the expected +7.58pp lift, and then ask: *which component is doing the work?*

The answer is uncomfortable for the standard story. **We cannot reject the null that either tool, on its own, contributes nothing.** Removing `python` (forcing the model to use only `validate`) yields 86.58%, slightly *above* baseline ($\Delta=+0.53\text{pp}$, well within the binomial standard error of $\sim 0.95\text{pp}$). Removing `validate` (leaving only `python`) yields 86.20% ($\Delta=+0.15\text{pp}$). But removing a single post-validation nudge—an 80-token user message that fires once after [VALIDATION PASS] to force `\boxed{\}` emission—drops accuracy by 4.25pp, an effect of more than two standard errors and qualitatively robust across re-runs.

The mechanism is mundane: without the nudge, the model after receiving [VALIDATION PASS] keeps invoking tools to “double-check,” exhausting `max_turns` and never reaching a clean boxed answer (Appendix B). The `validate` tool’s `verification_code` parameter is itself a Python snippet that the runtime executes, so “remove `python`” does not in fact remove Python execution from the loop; the model just routes its computation through `validate`’s argument. The two tools are most naturally read as redundant Python-

Table 1: GSM8K test accuracy along the training pipeline (no scaffolding).

Stage	0.8B	2B	4B
Pretrained base	52.92	69.98	83.62
+ OPD (1 epoch)	62.55	75.97	— (teacher)
+ OPD + GRPO variant	64.52	78.47	—

execution channels with one of them additionally exposing a PASS/FAIL judgment that the nudge can hook onto.

Our contributions:

1. A clean +7.58 / +10.24pp Harness Agent result on GSM8K with a 2B-OPD-GRPO student and the 4B teacher (§3).
2. A four-way ablation table (§4) showing that removing either tool individually changes accuracy by less than one standard error, while removing the post-PASS nudge drops it by 4.25pp ($> 2\sigma$). We propose a tentative decomposition— $\sim +4.3$ pp from forced termination, $\sim +3.3$ pp from the multi-turn framework, ~ 0 pp from each individual tool—under an additivity assumption we cannot fully verify (§4.1).
3. A reframing of “agent for math”: the productive levers, in this setup, are (a) some Python execution channel (which either tool supplies), (b) multi-turn retry budget, and (c) a forced termination mechanism. Tool *identity*, holding the Python channel constant, is not the lever (§5).

The paper is deliberately narrow in scope: one dataset (GSM8K, 1319 test samples), one model family (Qwen3.5), and one ablation axis (component removal, not variation). Section 7 discusses what this scope can and cannot support, including the one missing ablation cell (no-tool, nudge-only) that would distinguish “the Python channel is necessary” from “the nudge alone suffices.”

2 Background and Setup

2.1 The training pipeline (backbone, not the contribution)

The student backbone is **Qwen3.5-2B**, trained in two stages:

1. **On-Policy Distillation (OPD)** [Agarwal et al., 2023] from a Qwen3.5-4B teacher. Student samples a trajectory, teacher provides per-token reference logits, KL pushes the student toward the teacher distribution at each step. One full epoch on the GSM8K training split.
2. **GRPO variant** with two modifications: Dr.GRPO [Liu et al., 2025] (removes per-sequence length and within-group reward standardization), and DAPO Clip-Higher [Yu et al., 2025] (asymmetric PPO clip with $\epsilon_{lo} = 0.2$, $\epsilon_{hi} = 0.28$ to allow upside ratio updates). Trained 400 steps on a Boolean (boxed-correct) reward; we report the peak-checkpoint (ckpt-400) score.

Baseline pretrained, OPD-only, and OPD+GRPO numbers are summarized in Table 1. The 0.8B model is trained identically and reported for scale context but is not the focus.

The 2B+OPD+GRPO student closes 62% of the 2B-to-4B gap on raw CoT, but stalls at 78.47%—within ~ 5 pp of the 4B teacher. Diagnostic measurements of the token-level Top-K overlap between student and teacher distributions ($m_{\text{overlap}} = 0.71$ on held-out completions) suggest the student is at or near the capacity ceiling reachable from this teacher via on-policy distillation.

2.2 Harness Agent

The harness exposes two tools to the model through the standard OpenAI tool-calling interface:

- `python(code: str) -> str`: Run the code in a subprocess (5s timeout, only `math` builtin) and return stdout. Used to compute a candidate answer via the *forward* formulation.
- `validate(answer: str, verification_code: str) -> str`: Run `verification_code` (also Python, same sandbox), parse its last printed number, and compare to `answer`. Return `[VALIDATION PASS]` iff they match, else `[VALIDATION FAIL]`. The system prompt is explicit that `verification_code` must verify by *inversion* (plug `answer` back into the problem’s stated constraints and check), not by re-running the same forward formula.

Two control-flow patches sit on top of the tool layer:

- **Post-PASS nudge.** When a `validate` call returns `[VALIDATION PASS]`, the loop appends a single short user message: “*STOP calling tools. Output \boxed{<number>} on its own line.*” This fires exactly once per trajectory.
- **Coda.** If `max_turns` (=10) is exhausted without a boxed emission, a final user message forces a single boxed-only completion.

The system prompt walks through one worked example (Josh’s-house-profit, a classic “verify the forward formula instead of the constraint” trap) and demands inversion-style verification.

2.3 Evaluation protocol

All accuracy numbers below are on the full GSM8K test split (1319 problems), evaluated against gold via numeric tolerance ($|p - g| < 10^{-4}$ after stripping commas). The agent runs through vLLM 0.21 with the merged Qwen3.5-VL-base + trained-LM weights (the text-only checkpoint cannot be served by stock vLLM due to a hybrid-attention limitation). Sampling: temperature 0.7, top-p 1.0, max-tokens 2048, max-turns 10. Concurrency 128. Each ablation in §4 is a single full-1319 run; sampling noise is bounded by $\sqrt{p(1-p)/n} \approx 1.0$ pp in this regime.

3 Main Results

Table 2: Effect of the Harness Agent across student and teacher.

Model	base CoT	+Harness	Δ
2B + OPD + GRPO	78.47	86.05	+7.58
4B teacher	83.62	93.86	+10.24

Two observations:

1. A 2B model with scaffolding outperforms the 4B teacher’s raw CoT ($86.05 > 83.62$). The scaffold does not depend on the teacher’s strength.
2. The lift is larger for the stronger model (+10.24 on 4B vs +7.58 on 2B). The standard interpretation would be “the smarter model uses tools better”; we revisit this in §5.

Auxiliary metrics for 2B+Harness on the 1319 test set:

- Validation rate (a `validate` call returned PASS): 92.27%

- Acc conditional on validated: 86.69%
- Acc conditional on unvalidated: 78.43%
- Trajectories using `python` at least once: 68.4%
- Trajectories using `validate` at least once: 96.0%

The 96.0% `validate`-usage rate vs 68.4% `python`-usage is the first hint that “the validator is doing the heavy lifting”—but §4 shows this hint is misleading.

4 Ablation: What Drives the Lift?

We construct three orthogonal ablations on the 2B+OPD+GRPO+Harness setup, each removing exactly one component while leaving the other two intact:

- **–python:** Only `validate` is exposed. The model’s only Python channel is the `verification_code` parameter.
- **–validate:** Only `python` is exposed. No inversion gate; the model emits `\boxed{}` whenever it chooses.
- **–nudge:** Both tools present; the post-PASS nudge is suppressed. After `[VALIDATION PASS]`, the model is free to continue calling tools.

All other parameters held constant. Table 3 is the central result of the paper.

Table 3: Component ablation on 2B+OPD+GRPO. All numbers on the full GSM8K test split ($n = 1319$).

Configuration	Acc	Δ vs Harness	Val rate	py used	val used
Base CoT (no scaffolding)	78.47	−7.58	—	—	—
Harness (full)	86.05	—	92.3	68.4	96.0
– python	86.58	+0.53	94.0	0.0	95.3
– validate	86.20	+0.15	0.0	10.9	0.0
– nudge	81.80	−4.25	89.5	63.2	91.0

The binomial standard error at $p \approx 0.86$, $n = 1319$ is $\sqrt{p(1-p)/n} \approx 0.95\text{pp}$. We use 1σ as our criterion for “within noise.” Reading row by row:

- **Removing python does not hurt.** $\Delta = +0.53\text{pp}$ is below 1σ . With only `validate`, the model still has Python execution (via `verification_code`); the only thing lost is a redundant compute pathway. The slight non-significant uptick is consistent with (but does not establish) the model concentrating attention on the inversion-verification frame.
- **Removing validate does not hurt either.** $\Delta = +0.15\text{pp}$, well below 1σ . With only `python`, the model emits `boxed` without a PASS gate, and never validates (val rate = 0). Strikingly, only 10.9% of trajectories even *call* `python`; the model defaults to its CoT pathway and the agent format just provides longer context to think in.
- **Removing the nudge hurts substantially—4.25pp ($> 2\sigma$).** All other components intact, `validate` still usable, but the post-PASS user message is suppressed. The trajectories now show the failure mode the nudge was designed to prevent: after receiving `[VALIDATION PASS]`, the model keeps invoking `python` and `validate` to “make sure,” exhausts the 10-turn budget, and is then forced into a degraded coda completion (worked trace in Appendix B).

4.1 Decomposition of the +7.58pp lift

Treating the ablation table as a Shapley-style decomposition over the three components (tools-collectively, nudge, multi-turn framework), and using `-nudge = 81.80` as the “agent framework but no termination” anchor:

$$\begin{aligned}\Delta_{\text{nudge}} &= 86.05 - 81.80 = +4.25 \text{ pp} \\ \Delta_{\text{framework}} &= 81.80 - 78.47 = +3.33 \text{ pp} \\ \Delta_{\text{python alone}} &\approx 0, \quad \Delta_{\text{validate alone}} \approx 0\end{aligned}$$

The “framework” delta absorbs everything that the multi-turn agent format gives the model relative to a single-shot CoT prompt: more tokens (the trace can grow to $\sim 2\text{k}$ tokens of reasoning), retries on tool errors, and the worked-example pedagogy in the system prompt. The “nudge” delta is purely a control-flow effect on top of that—it does not give the model new capabilities, it merely tells it when to stop. **Caveat:** this decomposition assumes the two effects are additive. They need not be: the nudge would have nothing to trigger off without a discrete success event (`-nudge` with full tooling = 81.80%, but we have not tested `-nudge` with `-validate`, where no PASS event ever fires). We treat the decomposition as a working hypothesis, not a proven factorization.

4.2 Why does removing either tool individually not hurt?

The structural explanation is that `validate(answer, verification_code)` executes `verification_code` as Python under the hood. From the model’s perspective, calling `validate` with a non-trivial `verification_code` *is* a Python call, with the added side-effect of a final PASS/FAIL judgment. So `-python` does not in fact remove Python execution from the trajectory; the model just routes its computation through `validate`’s argument. Conversely, in `-validate`, the model still has `python` (though it uses it only 10.9% of the time, often relying on direct CoT to box the answer).

This explains why *either-but-not-both* removal is benign, but it does not prove the stronger claim that the tools are dispensable. We do not measure a no-tool baseline with nudge intact—the configuration {no python, no validate, nudge active}, in which the model would have to invoke the nudge from a non-existent PASS event. Without that cell, we cannot distinguish “any Python channel suffices” (the most parsimonious reading) from “a Python channel is necessary.” The reading that survives our data is the former *conditional on having a Python channel*; the absolute necessity claim is one well-designed run away.

5 Discussion

5.1 The validator’s marginal contribution is small, given the python tool

The validator’s nominal job is to catch wrong answers via inversion. In practice it does catch some—the `val-rate=92.3% / acc|validated=86.69% / acc|unvalidated=78.43%` split confirms validation is correlated with correctness. But the ablation shows that the *marginal* contribution of the validator-as-tool to final accuracy, on top of the python tool, is within sampling noise. What the validator does causally is *anchor* the trajectory to a discrete event (`[VALIDATION PASS]`) that the nudge can hook onto. We cannot say from this data whether a different anchor mechanism (e.g., a fixed-turn checkpoint, a model-confidence threshold) would substitute equally well; the prediction would be that any reliable PASS-like event the nudge can fire on should suffice, but this is a hypothesis our experiments do not test.

5.2 Why is the 4B’s gain bigger than the 2B’s?

A common explanation is that larger models “use tools better.” Our data suggest a different story for 4B+Harness’s +10.24pp: the 4B teacher’s `python`-call rate is much higher than the 2B’s (we report `val-rate`

99.92% and acc|validated 93.93% from a separate run), but the bulk of the marginal gain still goes through the same nudge-driven termination mechanism. The 4B is better at producing a *correct* candidate answer when given more tokens to think in, and so the same “force termination after PASS” lever pays off more. Tool capability is a side-effect, not a driver.

5.3 What we are not claiming

- We are not claiming validators or verifiers are useless in general. Validators that catch *categorically different* failure modes than the forward computation would behave differently in this ablation. The Qwen3.5-2B model rarely produces a candidate that passes inversion but is materially wrong; in domains where it would (e.g., constraint-satisfaction problems), the validator’s causal contribution may be much larger.
- We are not claiming this result generalizes to non-arithmetic domains. GSM8K’s solutions are short, numeric, and inversion-checkable. On open-ended reasoning, the trade-off changes.

5.4 Implications for scaffolding design

1. **Cheap wins on top of strong base models.** A ~ 80 -token nudge plus a tool that exposes Python is worth +7.58pp on a saturated 2B. Practitioners shipping math-capable LMs should prioritize this over more elaborate verifier designs.
2. **Negative claim about “smarter tools.”** Adding a second tool, or a more sophisticated verifier, may produce no measurable lift over {any Python channel} + {forced termination}—at least on benchmarks of this style. Ablation tables in future scaffolding papers should include the trivial “remove this tool” baseline.
3. **Termination as a first-class design dimension.** Most scaffolding papers treat the loop’s exit condition as a footnote (“emit boxed and we parse it”). The ablation here suggests it deserves its own row in the design table.

6 Related Work

Tool-augmented reasoning for math. Program-of-Thought [Chen et al., 2022] introduced executing Python as the *primary* reasoning path. ToRA [Gou et al., 2023] alternates between reasoning and Python and trains on tool-use trajectories. Self-verification approaches [Weng et al., 2022] check candidate answers via an auxiliary model or reversed problem. Our harness combines these ideas, then asks which ingredient matters.

On-policy distillation. Agarwal et al.’s OPD [Agarwal et al., 2023] is the immediate predecessor of our backbone. Liu et al.’s Dr.GRPO [Liu et al., 2025] removes length and within-group variance normalization in GRPO; DAPO [Yu et al., 2025] introduces asymmetric clipping. Our backbone combines all three.

Negative results for “agent” components. Concurrent work has noted that complex agent loops sometimes underperform CoT and that verifier choice is less important than expected in RL-from-feedback. Our finding sharpens this: in a specific clean setup, the verifier’s causal contribution is near zero, and the actual lever is a single control-flow token.

Effective Feedback Compute. Concurrent work [Zhang et al., 2026] formalizes a complementary scaling law: agent capability tracks not raw compute (tokens, tool calls, wall time) but the fraction of that compute that produces feedback the agent actually retains and uses to change its next decision. Their Oracle-EFC, normalized by per-task feedback demand, explains $R^2=0.99$ of the variance in agent performance across

synthetic and executable tasks, versus $R^2=0.33$ for raw token counts and $R^2=0.42$ for raw tool calls. Our paper offers a mechanism-level instance of this finding within a single fixed harness: the post-PASS nudge does not add tools, tokens, or model capacity, but it converts a previously-ignored [VALIDATION PASS] signal into a binding termination decision. In EFC vocabulary, the nudge is a *retention/use* operator: it raises the fraction of the trajectory that is informative, non-redundant, and acted upon, without changing the raw compute budget. The +4.25pp –nudge gap and the near-zero individual-tool gaps are exactly what an EFC-style account predicts—adding raw compute in the form of an extra tool does not move the needle; adding a control-flow primitive that increases retention does.

7 Limitations and Outlook

- **Missing ablation cell: no-tool with nudge.** We did not test the configuration {no python, no validate, nudge active}. This is the single cell that would distinguish “any Python channel is necessary” from “the nudge alone, on a multi-turn frame, suffices.” We expect the former (the model loses the ability to actually compute), but we have not verified it. This is the most important next experiment; a single ~15-minute run would either tighten or refute our central claim.
- **Single dataset.** GSM8K’s properties (short numeric solutions, easy inversion-checkability, ≤ 8 -step problems) drive much of the substitutability we report. MATH-500, SVAMP, and GSM-Plus would each test different facets. On harder benchmarks where forward-CoT is less reliable, the validator’s marginal contribution may grow.
- **Single model family.** All numbers are on Qwen3.5. The control-flow finding (nudge dominates) is unlikely to be Qwen-specific, but the absolute numbers will move. Qwen2.5-Math, Llama-3.2-Math, and Mistral-7B-Math would each be informative.
- **One ablation axis.** We ablate tool *removal*, not tool *variation*. A richer ablation (different validator designs, different nudge wordings, longer `max_turns`, a content-free vs. content-bearing nudge) would isolate the nudge effect more sharply.
- **Additivity assumed in the decomposition.** The +4.3pp / +3.3pp / ~0pp split assumes the three components act additively. The data are consistent with this but do not prove it.
- **Reward in-distribution.** The 2B backbone was trained on GSM8K specifically. We cannot disentangle how much of the Harness gain transfers OOD without rerunning the ablation on, e.g., a 2B that has only seen MATH.

8 Conclusion

Inference-time scaffolding lifts a 2B GSM8K model from 78.47% to 86.05%—a gain larger than the gap to the 4B teacher at base CoT. Decomposing the lift through clean ablations reveals that the gain comes almost entirely from two cheap ingredients: a multi-turn agent format that expands the model’s effective compute, and a single nudge that forces termination once a discrete success event fires. The two specific tools the harness offers—Python and an inversion validator—are mutually substitutable and individually contribute near zero. We argue the field has been over-crediting tool design and under-crediting termination control, and that future scaffolding work should isolate these effects.

References

- R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. *arXiv:2306.13649*, 2023.

- W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv:2211.12588*, 2022.
- Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. *arXiv:2309.17452*, 2023. ICLR 2024.
- Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv:2503.20783*, 2025.
- Y. Weng, M. Zhu, F. Xia, B. Li, S. He, S. Liu, B. Sun, K. Liu, and J. Zhao. Large language models are better reasoners with self-verification. *arXiv:2212.09561*, 2022. EMNLP 2023 Findings.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv:2503.14476*, 2025.
- X. Zhang, D. Wang, K. Xu, Q. Zhu, and W. Che. Scaling laws for agent harnesses via effective feedback compute. *arXiv:2605.29682*, 2026.

A Per-ablation auxiliary metrics

Config	n	Acc	Val rate	Acc val	Acc unval	py%	val%
Harness (full)	1319	86.05	92.3	86.69	78.43	68.4	96.0
– python	1319	86.58	94.0	88.87	50.63	0.0	95.3
– validate	1319	86.20	0.0	—	86.20	10.9	0.0
– nudge	1319	81.80	89.5	86.96	37.68	63.2	91.0

Notable: in `–python`, the *unvalidated* acc drops to 50.63% (the model has no fallback Python and gets unvalidated answers wrong). In `–nudge`, *unvalidated* acc drops to 37.68%—these are the trajectories that exhausted `max_turns` without ever boxing.

B Worked failure mode without nudge

A representative `–nudge` trajectory after [VALIDATION PASS]:

```
[turn 5] validate -> [VALIDATION PASS] (claimed=18, verified=18)
[turn 6] assistant: <tool_call><function=python><parameter=code>
    # Let me double-check by computing it a third way
    ...
[turn 7] assistant: <tool_call><function=validate>... (re-validates same answer)
[turn 8] assistant: <tool_call><function=python>...
...
[turn 10] coda: "Stop calling tools, output boxed now (validated)."
```

```
[turn 10] assistant: "\boxed{16}"  <- changed answer in the coda call
```

The model after PASS often re-checks until it accidentally talks itself out of the correct answer in the forced coda turn. The nudge cuts the trajectory off at turn 6 with [VALIDATION PASS received] STOP calling tools immediately. Output `\boxed{18}`., preserving the validated answer.